1 Real-time SSR corrections

The real-time engine consumes network corrections in *State Space Representation* (SSR): per-satellite, per-signal refinements to the broadcast orbit, clock, and hardware biases, streamed as RTCM 3.x messages (legacy 1057–1068 / 1240–1270) or as their IGS-SSR `4076` repackaging [1, 2]. Each SSR quantity refines *exactly one* term already present in the undifferenced code equation and the undifferenced phase equation of the Foundations document; none introduces a new state. This is what separates the precise real-time mode from broadcast-only operation: with SSR applied, $c\,\mathrm{d}t^i$ and $\boldsymbol{r}^i$ are centimetre-grade and the satellite biases B_s^i, b_s^i are *known* rather than absorbed into the ambiguity. The corrections in this document are the rover-side *consumer*: the same on-disk store is the sink for both a third-party IGS-RTS stream and the engine's own network estimator (the Network mode document), so the application math below is identical regardless of who produced the corrections.

Decoded records are cached per satellite in `products/SsrCorrectionStore` (a process-wide singleton for the live stream, or a caller-owned instance for the network producer's closed-loop test); the consumers (`model/ModelGeometry`, `corrections/GnssCorrections`, `measurement/MeasurementModel`) query the store and fall through to broadcast-only when it is empty, so every equation below collapses to the broadcast-only observation model in post-processing. The streaming and decoding plumbing (RTCM frame parse, the per-satellite mutex-guarded cache, the real-time push loop) belongs to the Real-time engine document; this document covers only the store-driven correction math, which is invariant to how records arrived. There is no `enable_ssr` flag: SSR application is implicit and store-driven.

1.1 Orbit correction: refines ρ_r^i

The orbit message carries a delta in the satellite's radial / along-track / cross-track (RAC) frame, anchored to the broadcast ephemeris of issue IODE. The frame is built from the broadcast position $\boldsymbol{r}^i_{\mathrm{brdc}}$ and velocity $\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}$ (`products/SsrCorrectionStore.cpp`):

$$\hat{\boldsymbol{e}}_a = \frac{\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}}{\|\dot{\boldsymbol{r}}^i_{\mathrm{brdc}}\|}, \quad \hat{\boldsymbol{e}}_c = \frac{\boldsymbol{r}^i_{\mathrm{brdc}} \times \dot{\boldsymbol{r}}^i_{\mathrm{brdc}}}{\|\boldsymbol{r}^i_{\mathrm{brdc}} \times \dot{\boldsymbol{r}}^i_{\mathrm{brdc}}\|}, \quad \hat{\boldsymbol{e}}_r = \hat{\boldsymbol{e}}_a \times \hat{\boldsymbol{e}}_c. \tag{1}$$

The RAC delta is propagated to the query epoch with its rate term and rotated into ECEF. With $\delta t = t - t_{\mathrm{SSR}}$ the age of the correction, $\boldsymbol{\delta}_{\mathrm{rac}} = (\delta_r, \delta_a, \delta_c)^{\mathrm{T}}$ its components and $\dot{\boldsymbol{\delta}}_{\mathrm{rac}}$ their rate,

$$\boxed{\boldsymbol{r}^i \;=\; \boldsymbol{r}^i_{\mathrm{brdc}} \;-\; \big[\,\hat{\boldsymbol{e}}_r \;\; \hat{\boldsymbol{e}}_a \;\; \hat{\boldsymbol{e}}_c\,\big]\big(\boldsymbol{\delta}_{\mathrm{rac}} + \dot{\boldsymbol{\delta}}_{\mathrm{rac}}\,\delta t\big).} \tag{2}$$

The *subtraction* follows the RTCM sign convention $\boldsymbol{\delta}$ = broadcast − precise (`products/SsrCorrectionStore.cpp`). The corrected $\boldsymbol{r}^i$ then enters $\rho_r^i = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ and the line-of-sight $\boldsymbol{e}^i_{\hat{r}}$ of the Foundations document unchanged: the orbit correction never touches an observation equation directly, only the geometry (the observation-geometry section) that feeds it. The delta is applied only when its IODE matches that of the broadcast ephemeris the baseline orbit was evaluated from (Equation (6)); on a live cutover the baseline position is deliberately re-referenced to the ephemeris *version* the SSR delta was issued against rather than dropped to broadcast grade (`model/ModelGeometry.cpp`, via `LiveEphemerisStore::computeSatellitePV` with the SSR IODE). Because the clock is queried against the same broadcast ephemeris, the broadcast

clock's *radial* component cancels consistently against Equation (4). The decoded RAC scale factors are fixed by RTCM 10403.3 (radial δ_r a 22-bit two's-complement field at 0.1 mm, along/cross at 0.4 mm, rates at 1 and 4 μm/s respectively; `rtcm/RtcmSsrDecoder.cpp`). The orbit record also carries a *reference point* for the corrected $\boldsymbol{r}^i$: legacy RTCM SSR corrects to the satellite antenna phase centre (APC), while IGS-SSR `4076` and the network product correct to the centre of mass (CoM) (`rtcm/RtcmSsrDecoder.cpp`). Under `realtime.honor_ssr_reference_point` (RTB-5; default OFF, byte-identical) the consumer records whether the applied overlay was APC-referenced (`ssr_orbit_apc_applied_` in `model/ModelGeometry.cpp`) so that `PppModel` suppresses its own satellite-PCO term on that geometry rather than double-counting the phase-centre offset the orbit correction has already resolved.

1.2 Clock correction: refines $-c\,\mathrm{d}t^i$

The clock message carries a quadratic polynomial in time-from-epoch, optionally augmented by a high-rate additive delta δC_{hr}. The total satellite-clock correction, in metres, is

$$\Delta C^i(t) \;=\; C_0 + C_1\,\delta t + C_2\,\delta t^2 + \delta C_{\mathrm{hr}}, \qquad \delta t = t - t_{\mathrm{SSR}}, \tag{3}$$

converted to seconds and *added* to the baseline clock (`products/SsrCorrectionStore.cpp`):

$$\boxed{c\,\mathrm{d}t^i \;=\; c\,\mathrm{d}t^i_{\mathrm{base}} \;+\; \Delta C^i(t),} \tag{4}$$

where $\mathrm{d}t^i_{\mathrm{base}}$ is the precise CLK sample (post-processing) or the broadcast clock polynomial (live rooftop mode, `products/LiveEphemerisStore`). This term enters both undifferenced observation equations through $-c\,\mathrm{d}t^i$, identically on code and phase. The high-rate clock is additive *on top of* the polynomial, not a replacement (`corrections/GnssCorrections.cpp`); the C_2 field is a 27-bit two's-complement quantity at 2×10^{-8} m/s^2.

Split-cadence fast clocks. A third, structurally different way for the stored clock to move is a *fast tick* (`SsrCorrectionStore::pushFastClock`), which the network producer can run between full solves on decimated-out epochs. A fast tick re-bases exactly three fields of an *existing* clock record, namely C_0 and the two epoch stamps, so (3) is thereafter evaluated at the fast δt while retaining the slow era's C_1 and C_2:

$$\Delta C^i(t) = C_0^{\mathrm{fast}} + C_1\,(t - t_{\mathrm{fast}}) + C_2\,(t - t_{\mathrm{fast}})^2. \tag{5}$$

Two omissions are what make this safe for a consumer, and both are contract, not accident. The discontinuity counter is *not* advanced: a fast row is a continuation of the same clock, not a new arc, so it must not trip the ambiguity reset of the discontinuity paragraph below at the fast cadence. And the shared per-satellite SSR header is *not* restamped, so the code-bias, phase-bias, URA and slant-iono channels do not inherit the fast clock's age off one refresh. The method is also refresh-only: it returns false rather than creating a record, so a consumer can never be handed a satellite that exists on the fast channel alone with no bias set to go with it. It is wired both in-process (`filter/NetworkFastClock`) and on replay of an on-disk product (`products/NetworkProductIO`), and is default OFF (`network.fast_clock.enable`, which the loader also disarms when the run's posture leaves no epoch for a tick to ride).

1.3 IOD gate at correction-set boundaries

Orbit and clock corrections are deltas against a *specific* broadcast ephemeris, so they are applied only when the SSR record's IODE matches that of the broadcast ephemeris from which the baseline orbit/clock was evaluated:

$$\text{apply} \iff \big(\text{IODE}_{\text{brdc}} < 0\big) \ \vee \ \big(\text{IODE}_{\text{SSR}} < 0\big) \ \vee \ \big(\text{IODE}_{\text{brdc}} = \text{IODE}_{\text{SSR}}\big), \tag{6}$$

the gate being permissive when either side is unknown (`SsrCorrectionStore::iodeMatches`, `products/SsrCorrectionStore.hpp`). A combined orbit+clock record (RTCM 1060/1066, IGS-SSR sub-type 3) is preferred over separate messages because it guarantees a single shared IODE (`products/SsrCorrectionStore.cpp`). Since RTF-2 the two overlays gate on *separate* anchors: the orbit delta on the record's orbit `iode` (`model/ModelGeometry.cpp`), the clock polynomial on a distinct `clock_iode` field (`corrections/GnssCorrections.cpp`). For a standard stream `clock_iode` inherits the orbit anchor in `getOrbitClock`, so the two coincide and (6) reads as the single gate written; only a `brdc_rts` producer, which stamps its own clock IODE, makes them differ. At a correction-set boundary (the provider issuing a new ephemeris reference, hence a new IODE), the gate (6) simply declines the mismatched record until the matching broadcast ephemeris is in hand, avoiding a discontinuous jump in $\boldsymbol{r}^i$ or $c\,\mathrm{d}t^i$. Records older than `max_age_s` (`processing.ssr_max_age_s`, default 30 s) are treated as absent by default. The RTA-9(b) stale-hold ladder relaxes this binary clock gate into three rungs (`SsrCorrectionStore::classifyClockAge`): with `processing.ssr_clock_stale_hold_factor` $f > 1$ a clock aged in (`max_age_s`, $f \cdot$ `max_age_s`] is *StaleHeld*, still applied as the extrapolated $C_0 + C_1\,\delta t$ of (3), with a matching URA-staleness R inflation on the consuming rows (`corrections/GnssCorrections.cpp`, `measurement/MeasurementModel.cpp`), while a record beyond the hold window is *Rejected*. The default $f = 0$ collapses the ladder back to the plain fresh-or-absent gate (byte-identical).

1.4 Satellite biases: refine $-B^i_s$ and $-b^i_s$

SSR delivers *observable-specific biases* (OSB): one value per (satellite, RINEX-3 signal) pair, in metres. They are applied as a *known* additive correction to the modelled observable, with no state and no Jacobian row (`measurement/MeasurementModel.cpp`). The code OSB refines B^i_s in the undifferenced code equation:

$$\boxed{P^i_{r,s} = \rho^i_r + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t^{(g)}_r + T^i_r + \mu_s I^i_{r,1} + B_{r,s} \underbrace{- B^i_s}_{\text{SSR OSB}} + \varepsilon^P,} \tag{7}$$

and the phase OSB refines b^i_s in the undifferenced phase equation:

$$\boxed{\Phi^i_{r,s} = \rho^i_r + c(\mathrm{d}t_r - \mathrm{d}t^i) + c\,\delta t^{(g)}_r + T^i_r - \mu_s I^i_{r,1} + \lambda_s N^i_{r,s} + b_{r,s} \underbrace{- b^i_s}_{\text{SSR OSB}} + \varepsilon^\Phi.} \tag{8}$$

The bias decoded from the stream is the satellite bias *present in* the observation, so it is removed from the clean model prediction with a $+1$ sign convention (`kSsrCodeBiasSign` = `kSsrPhaseBiasSign` = $+1$, `measurement/MeasurementModel.cpp`); the predicted observable accumulates it into the predicted observable as `ssr_bias_term` (`MeasurementModel.cpp`), summed alongside the receiver clock, troposphere, ionosphere, estimated code/phase biases and ambiguity. Biases are queried per *raw* signal only: an ionosphere-free composite key such as `C1W_C5X` returns zero (code) or `nullopt` (phase) and is skipped, consistent with the uncombined parameterisation. The code OSB is a

14-bit field at 0.01 m, the phase OSB a 20-bit field at 0.1 mm (`rtcm/RtcmSsrDecoder.cpp`). The application path is *ungated* by default: `ssr_bias_term` loads whenever the store carries the exact RINEX signal key. An optional A5 staleness gate (`processing.enable_ssr_bias_max_age`, default OFF) rejects a bias whose obs-vs-record age exceeds `processing.ssr_bias_max_age_s` (default 30 s), holding the last value over sub-limit gaps out to `ssr_bias_stale_hold_factor` × that age rather than manufacturing a model step (`measurement/MeasurementModel.cpp`). The corollary of the application path, avoiding a redundant estimated bias on the same row, is the subject of the next subsection.

1.5 Satellite-bias STATE gate vs. SSR code biases

The uncombined model can also *estimate* a per-(satellite, signal) code-bias state (`estimation.estimate_sat_biases`; see the state-vector section). That estimated state and a known SSR code OSB both enter the code row with unit coefficient and opposite physical roles, so allowing both for the same (i, s) would be doubly redundant: the stream already fixes the satellite code bias, and a free state collinear with it is near-unobservable and merely soaks up noise. State allocation therefore consults *both* correction sources before creating a sat-bias state (`MeasurementModel::ensureRequiredStates`):

$$\text{create sat-bias}(i,s) \iff \neg\,\text{hasBIA}(i,s) \;\wedge\; \neg\,\text{hasSsrCB}(i,s) \;\wedge\; \text{pivotCovers}(i,s), \tag{9}$$

where hasBIA tests the static `.BIA` product (`PppModel::hasBiasEntry`) and hasSsrCB tests the live stream (`SsrCorrectionStore::hasCodeBias`, surfaced as `ssrCoversCodeSignal`). Two design points make the gate correct:

- **Presence, not magnitude.** `hasCodeBias` is an exact-key *presence* test on the per-signal map, distinct from `getCodeBias` $\neq 0$: a provider's present-but-zero entry still defines that signal's datum frame, so the estimator must not re-estimate it. The two queries are required to agree exactly so a covered signal is never both estimated *and* SSR-corrected on one row.
- **Per-signal, not global.** IGS streams are partial (e.g. Galileo E1 covered, E5a/E5b not). The gate keys on the exact (i, s) pair, so a signal the stream does *not* carry keeps its estimated state, its only available code correction, while a covered sibling on the same satellite is deferred to the OSB. A strict per-epoch prune (`pruneSatBiasStatesExcept`) drops the state when SSR coverage arrives mid-arc and re-creates it (warm-restarted from the stashed cache) if coverage later disappears.

In post-processing the live store is empty, so hasSsrCB is uniformly false and the gate reduces to the static-`.BIA` test alone.

Network-producer exemption. When the engine *is* the network estimator (`ProcessingMode::PPP_RTK_Network`), the roles invert: it *estimates* the satellite code/phase biases as states to emit as a product (the Network mode document). Importing SSR biases there would double-count and render those very states unobservable, so both the `ssr_bias_term` application path (7) and the hasSsrCB branch of the state gate (9) are forced off on the producer side (`ssrCoversCodeSignal` returns false). The producer's own emitter (`products/NetworkProductEmitter.cpp`) then pushes the estimated clock and code/phase biases into a *separate* store instance, closing the loop against a held-out user consumer.

Integer recoverability via consistent phase biases. Removing b_s^i is precisely what restores the integer property of $N_{r,s}^i$ promised in the phase observation equation of the Foundations document. Once the satellite phase bias is known, the residual non-integer part of the ambiguity is the receiver phase bias $b_{r,s}$ *alone*, which cancels under between-satellite single differencing $\Delta\cdot$ (the differencing section), leaving an integer single-difference ambiguity admissible to AR. This is the operational content of a *consistent* phase-bias product: the provider must generate b_s^i on the same network datum whose integer property the rover then exploits. The phase-bias message therefore carries three AR-critical metadata fields per signal (`rtcm/RtcmTypes.hpp`): an integer-preservation indicator, a two-bit wide-lane integer indicator, and a four-bit *discontinuity counter*.

Discontinuity handling. An incremented discontinuity counter signals that the provider's phase bias jumped between correction sets, breaking the integer connection of that arc. The engine treats this as an ambiguity-arc boundary: the affected $N_{r,s}^i$ is reset before the Kalman update (`realtime/RealTimeEngine.cpp`, via `StateManager::noteSsrDiscontinuity`), so the post-jump phase is re-initialised rather than fixed against a stale integer offset. This is the phase-bias analogue of the orbit/clock IOD gate (6): both protect state continuity across correction-set boundaries, one by declining a mismatched delta, the other by resetting the dependent ambiguity.

1.6 GLONASS

GLONASS SSR is decoded and stored on the same footing as the CDMA constellations: the legacy block (orbit 1063, clock 1064, code bias 1065, combined 1066, URA 1067, high-rate clock 1068, phase bias 1266) and the IGS-SSR `4076` subtypes in the GLONASS message-indicator band (IM = 41–47) both map onto the identical orbit/clock/bias structs (`rtcm/RtcmSsrDecoder.cpp`). The orbit, clock and bias *equations* above apply unchanged; nothing in Equations (2), (4), (7) and (8) is constellation-specific.

The one GLONASS-specific subtlety is on the *time tag*, not the correction math. GLONASS SSR carries its content epoch as a time-of-day field (DF389, seconds since the start of the GLONASS UTC(SU)+3 h day) rather than a GPS time-of-week. The decoder promotes it to the GPS scale through `TimeUtils::glonassTodToGpsTime` (`rtcm/RtcmSsrDecoder.cpp`); without that promotion the SSR epoch would land at $\{0, 0\}$, the freshness check (6)'s sibling `max_age_s` gate would declare every GLONASS record stale, and the satellite would silently fall back to broadcast grade. Once promoted, the per-record content epoch drives $\delta t = t - t_{\mathrm{SSR}}$ in Equations (2) and (4) exactly as for GPS. Whether the resulting GLONASS observable then *contributes* to a between-satellite-differenced solution is a separate, estimator-side opt-in (its FDMA per-channel receiver delay does not cancel in SD); see the differencing section.

1.7 Ionosphere: $\mu_s I_{r,1}^i$ off-wire, via the slant channel

For completeness, and to forestall a natural misreading of the term: although the SSR standard defines a global VTEC spherical-harmonic message (IGS-SSR IM = 201) and regional STEC products, *neither is decoded by this engine*. The IGS-SSR classifier rejects IM = 201 as unsupported (`rtcm/RtcmSsrDecoder.cpp`), so nothing on the RTCM wire writes the slant-iono state $I_{r,1}^i$. On that wire the ionospheric term $\mu_s I_{r,1}^i$ is therefore informed only by (i) the per-satellite iono *state* estimated by the filter itself and (ii) the optional GIM/IONEX deterministic prior (`atmosphere/GlobalIonosphereMap`, an independent VTEC→STEC mapping unrelated to the SSR stream).

The atmosphere loop is, however, closed *off* the RTCM wire, through the same `SsrCorrectionStore` the clock and bias overlays use. The store carries a dedicated undifferenced

slant-iono channel (`SsrSlantIono`, via `SsrCorrectionStore::pushSlantIono` / `getSlantIono`): the network producer's emit seam (`filter/NetworkEmitStore`) evaluates its co-estimated SH-VTEC field (the co-estimated VTEC field) at the consumer's pierce points and pushes a per-LOS slant delay and 1σ, and the consumer (`RealTimeEngine::processEpoch`) folds each fresh, same-epoch record into an ionosphere-weighted (BLUP) pseudo-observation on the matching slant-iono state (`KalmanFilter::applyPseudoObservation`), exactly the iono-weighted constraint that makes single-epoch AR feasible. The fold is gated by `closed_loop.consume_network_iono` (default OFF, so no rows are built and the update is byte-identical) and admits a value only when the producer refreshed it this epoch (`max_age` $= \frac{1}{2}$ the epoch spacing), so a satellite the producer did not re-supply floats rather than inheriting a stale delay.

Three properties of the channel are worth stating because they are inherited from the field, not chosen here. The supply is GPS and Galileo only: GLONASS is excluded from the SH field itself, so a consumer's GLONASS slant states simply float. When a second shell is armed the pushed value is the *sum* over shells, each evaluated at its own pierce geometry, so the consumer is layer-agnostic by construction. And the per-LOS σ is the field's own representativeness floor (the SH representativeness sigma), optionally root-sum-squared with the SH coefficients' *formal* σ propagated through the same weighted basis and additionally gated on fresh OSB coverage for that satellite, under `network.iono_supply_formal_sigma` (default OFF, byte-identical). On the consumer side the rows are not independent: they are folded as a per-constellation common-mode-free block with exactly the noise structure of the GIM iono-weighting covariance, and frozen BSSD pivots (and any zero-variance slant state) are excluded from it for exactly the reason given there: the common-mode block exists to absorb the datum offset of the *remaining* rows, so the datum satellite must not appear as a measurement.

What remains open is a *standardised* on-wire STEC product: the in-process channel is a closed-loop store hand-off, not an IGS-SSR/RTCM message either side can yet decode from a third-party stream.

1.8 Provenance and code pointers

Both transports map onto the same internal correction structs (`rtcm/RtcmTypes.hpp`) through a single decoder (`rtcm/RtcmSsrDecoder.cpp`). Legacy RTCM uses per-constellation six-message blocks (orbit, clock, code bias, combined orbit+clock, URA, high-rate clock) plus a separate phase-bias block (1265–1270); IGS-SSR `4076` repackages the same payloads under a message-indicator (IM) whose *tens* place ($\lfloor \mathrm{IM}/10 \rfloor$) selects the constellation (GPS $2x$, GLONASS $4x$, Galileo $6x$, QZSS $8x$, BeiDou $10x$, SBAS $12x$) and whose *ones* place (IM mod 10) selects the subtype (1=orbit, 2=clock, 3=combined, 4=high-rate, 5=code bias, 6=phase bias, 7=URA). Note that this per-block subtype ordering differs from the legacy SSR block ordering, so the legacy mapping is not reused (`RtcmSsrDecoder.cpp`). VTEC (IM 201) and NavIC (IM 141–147) are rejected by the message-indicator classifier *before* the header is parsed, so they return `ok = false` with an empty header. This is the decode-side reason the ionospheric term carries no *on-wire* SSR contribution (the off-wire `SsrSlantIono` store channel of the ionosphere subsection is unaffected).

The invariant correction definitions of Equations (2), (4) and (6) live in the static, deterministic `applyOrbitToPosition` / `applyClockToBias` / `iodeMatches` members of `products/SsrCorrectionStore`, which is the seam this document documents. The stateful streaming layer (the mutex-guarded per-satellite cache with wall-clock freshness gating, the `RealTimeEngine::feedRtcmFrame` push loop, and the obs↔SSR synchronisation that bounds how far the obs stream may run ahead of lagging corrections) belongs to the Real-time engine document and is kept separate so the correction math can be reasoned about (and tested) in isolation. The one user-

facing SSR-related estimator config is the warm-up gate `estimation.defer_until_corrections_ready` (`config/GnssConfigLoad.cpp`), which defers estimation until a solvable floor of satellites is broadcast-plus-SSR centimetre-grade, so the filter seeds code and phase together rather than cold-starting a contaminated partition on metre-grade broadcast clocks.

Alongside the correction records the store also carries a per-epoch producer *heartbeat* (`NetworkProducerStatus`), which is what lets a consumer distinguish "the producer withheld this epoch" from "the producer died": a withheld epoch still writes its status line even though it publishes no corrections. The wire token set is deliberately forward-compatible: an unrecognised token from a newer producer maps to `State::Unknown` and is surfaced, never fatal (`SsrCorrectionStore::parseState` / `stateName`). One of the tokens now reports a withhold driven by the health of the producer's own SH-VTEC field; the *policy* behind that withhold (what engages it, what it suppresses, and what it does not) belongs to the Network mode document and is not restated here.

References

[1] International GNSS Service Real-Time Working Group. *IGS State Space Representation (SSR) Format, Version 1.00*. Format specification. RTCM message 4076 IGS-SSR sub-types. International GNSS Service (IGS), 2020.

[2] *RTCM Standard 10403.3: Differential GNSS Services – Version 3*. Special Committee No. 104; State Space Representation messages 1057–1068, 1240–1270, 4076. Radio Technical Commission for Maritime Services. Arlington, VA, USA, 2016.